

# Infrastructure-Free Autonomous UAV Landing in Unknown Terrain via Semantic Detection, Evidence-Grid Fusion, and Abort-Capable State Control

Author Name

Affiliation

Email

**Abstract**—Autonomous landing of unmanned aerial vehicles (UAVs) in unknown, unstructured terrain remains an open challenge, particularly when fiducial markers, pre-surveyed maps, or ground infrastructure are unavailable. This paper presents a complete perception–fusion–control pipeline for infrastructure-free UAV landing. The system couples a real-time YOLO-based multi-class object detector with a configurable safety ontology that maps detected classes to *safe*, *unsafe*, and *neutral* semantics. Detections are projected into world coordinates using onboard visual odometry and accumulated into a two-dimensional evidence grid with temporal decay, enabling the system to track evolving terrain conditions. A zone selector extracts contiguous safe regions from the grid, enforcing minimum-area constraints and spatial margins from hazards. An eight-state finite-state machine orchestrates the complete landing sequence (survey, evaluate, search, approach, align, descend, landed) with an explicit *abort* transition that triggers when dynamic intrusions contaminate a previously committed landing zone. The pipeline is validated in an offline PyBullet simulation with procedurally generated terrain and dual-viewpoint video rendering, and on real aerial imagery using a pre-trained YOLO detector. Across five randomized terrain seeds, the system achieves a 100% successful landing rate with a mean mission time of 35.4 s and mean grid convergence in 17.3 s. Abort behavior is validated with zero-latency intrusion detection across all seeds. Integration with Gazebo and the PX4 flight stack under ROS 2 is demonstrated at the launch-configuration level, and a concrete hardware deployment path using the Holybro X500 V2 airframe with a Pixhawk 6C autopilot is outlined.

**Index Terms**—UAV, autonomous landing, YOLO, evidence grid, safety mapping, finite-state machine, PyBullet, PX4, infrastructure-free

## I. INTRODUCTION

The ability to land safely without relying on prepared infrastructure is a prerequisite for deploying UAVs in disaster response, remote surveying, package delivery to uncontrolled sites, and military operations in denied environments. Traditional approaches to autonomous landing either assume a cooperative target (an ArUco marker, a helipad, or an infrared beacon) or require a pre-built map of the landing area. These assumptions collapse in genuinely unknown terrain: the vehicle must perceive, evaluate, decide, and control in a closed loop, using only its onboard sensors.

Recent surveys [1], [2] catalogue the breadth of vision-based landing research, noting a clear trend from marker-dependent systems toward natural-terrain methods that exploit deep learning for scene understanding. However, most existing pipelines address only a subset of the full problem. Semantic segmentation approaches [3], [4] classify pixels but rarely accumulate evidence over time or handle dynamic hazards. Grid-based methods [5] build spatial maps but often lack a systematic controller for the multi-phase landing sequence. Behavior-tree or FSM controllers [6], [7] manage mission logic but are typically demonstrated with cooperative markers rather than semantically derived zones.

This paper presents a unified pipeline that bridges these gaps. Its contributions are:

- 1) A **YOLO-to-safety-ontology mapping** that translates multi-class object detections into a three-tier semantic label scheme (safe, unsafe, neutral), configurable via YAML without retraining.
- 2) A **decaying evidence grid** that fuses projected detections over time, enabling the system to “forget” stale observations and respond to moving obstacles.
- 3) A **zone selector** that extracts contiguous safe regions with minimum-area and hazard-margin constraints, going beyond single-point selection.
- 4) An **eight-state abort-capable finite-state machine** that governs the full survey-to-touchdown sequence with explicit intrusion-triggered abort.
- 5) An **offline PyBullet simulation** with procedural terrain and dual-viewpoint rendering for repeatable evaluation without ROS or flight-stack dependencies.
- 6) A **ROS 2 / Gazebo / PX4 integration path** demonstrated at the configuration and launch level, with a concrete hardware deployment plan for the Holybro X500 V2 and Pixhawk 6C.

The remainder of this paper is organized as follows. Section II reviews related work. Section III details the system architecture. Section IV describes the simulation framework. Section V presents experimental results. Section VI outlines the hardware integration path. Section VII discusses limita-

tions, and Section VIII concludes.

## II. RELATED WORK

We organize related work into three pillars: perception, spatial fusion, and mission control, then discuss simulation environments and integrated systems.

### A. Vision-Based Perception for Landing Safety

Early vision-based landing systems relied on fiducial markers or cooperative targets [1]. The shift toward marker-free operation has been driven by deep learning. Yang et al. [8] demonstrated monocular SLAM for mapping unknown terrain and identifying flat regions, but without semantic class labels. More recently, Tan et al. [9] proposed VisLanding, which predicts depth and surface normals from monocular images and adds a binary safe/unsafe segmentation head. Gonzalez-Trejo et al. [10] addressed urban landing among crowds by building density-based risk maps from segmented human detections.

Serrano and Bandala [11] applied YOLO-family detectors to terrain classification, demonstrating that detection-based (as opposed to dense-segmentation) approaches offer a favorable speed-accuracy tradeoff for onboard deployment. Our work extends this direction by mapping YOLO class outputs through a configurable safety ontology and fusing them temporally, rather than making per-frame decisions.

Bektash et al. [12] and Kinahan and Smeaton [4] explored lightweight CNNs and semantic segmentation for whole-image or pixel-level safety classification on resource-constrained platforms. These confirm the feasibility of onboard semantic assessment but do not address temporal accumulation or dynamic obstacle handling.

### B. Spatial Fusion: Occupancy and Evidence Grids

Elfes [13] introduced probabilistic occupancy grids, which remain foundational for spatial reasoning in robotics. Scherer et al. [14] extended log-odds grids to MAV obstacle avoidance in 3D. For landing specifically, Secchiero et al. [5] fused BiSeNet segmentation with stereo disparity into a 2D binary occupancy map, then evaluated candidate zones by slope, flatness, and roughness. Their system avoids full 3D reconstruction by working directly in a 2D grid, a design choice we share.

Gross et al. [15] (SafeLand) build a Bayesian semantic ground map with temporal decay from neural segmentation, closely paralleling our evidence grid. The key distinction is that SafeLand uses dense pixel-level segmentation, whereas we use bounding-box-level object detection (YOLO), which is computationally lighter and naturally maps to a class-based safety ontology.

De la Torre-Vanegas et al. [16] proposed a similar risk-map framework with altitude-dependent revalidation during descent, using dense segmentation and temporal filtering.

### C. Mission Control: FSMs and Behavior Trees

Finite-state machines are the dominant control abstraction for multi-phase landing. Palafox et al. [6] implemented an FSM with takeoff, tracking, and landing states, including re-localization on target loss. Recio et al. [7] used a hierarchical state machine under ROS 2 for landing on a moving rover. Gross et al. [15] employ a behavior tree that pauses or reroutes on obstacle appearance.

Our FSM extends these patterns with an explicit eight-state design that includes ABORT as a first-class state, triggered by evidence-grid contamination of a committed zone, and ALIGN as a dedicated pre-descent stabilization phase.

### D. Simulation Environments

Panerati et al. [17] introduced gym-pybullet-drones, a PyBullet-based multi-agent quadrotor simulator for reinforcement learning. Song et al. [18] presented Flightmare, decoupling physics and rendering for high-throughput training. Tovanche-Picón et al. [19] validated vision-based landing in Unreal Engine + AirSim with synthetic urban crowds. Gazebo with PX4 SITL [20] remains the standard for end-to-end UAV software-in-the-loop testing under ROS 2 [21].

Our PyBullet simulation provides a self-contained, ROS-free evaluation path with procedural terrain, dual-camera rendering, and configurable YAML scenarios, complementing the heavier Gazebo/PX4 integration.

### E. Closest Integrated Systems

The most directly comparable prior works are Secchiero et al. [5], who combine segmentation, stereo geometry, and grid fusion but lack temporal decay and abort logic; and Gross et al. [15], who integrate Bayesian mapping with a behavior tree but use dense segmentation rather than object detection. Our system occupies a distinct point in the design space: lightweight YOLO detection, decaying evidence grid, and abort-capable FSM, unified in a single pipeline with both offline and ROS 2 deployment paths.

## III. SYSTEM ARCHITECTURE

The system is organized as a four-stage pipeline: *Perception* (Sec. III-A), *Pose Estimation* (Sec. III-B), *Fusion and Zone Selection* (Sec. III-C), and *Landing Control* (Sec. III-D). Fig. 1 shows the complete data flow.

### A. Perception: YOLO Detection and Safety Mapping

The perception stage converts raw camera images into semantically labeled detections suitable for fusion.

1) *Multi-Class Object Detection*: We use the Ultralytics YOLO framework [22], [23] as the detection backbone. The detector processes downward-facing (nadir) camera frames and produces bounding boxes with class labels and confidence scores. Both pre-trained COCO weights and a custom terrain-trained model (with classes including grass, rock, water, tree, building, car, person) are supported. The detector is wrapped in a `LandingPadDetector` class that handles model loading, inference stride control (to skip frames for computational

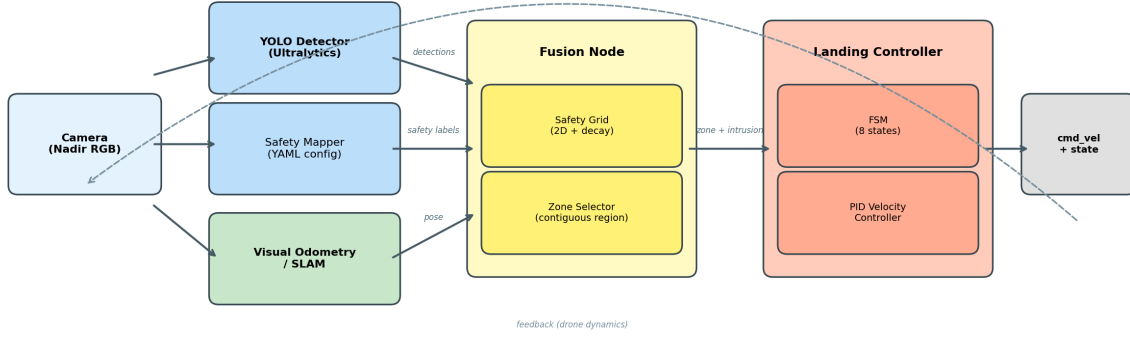


Fig. 1. System architecture showing the four-stage pipeline from camera input to velocity commands. JSON-serialized messages connect nodes in the ROS 2 deployment; in the PyBullet offline mode, the same Python objects are passed directly.

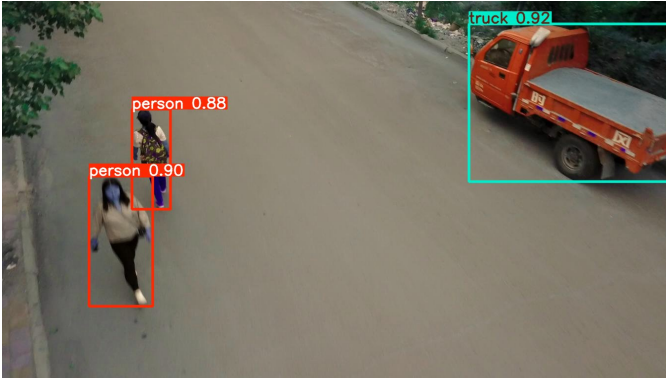


Fig. 2. Pre-trained YOLO detection on a real aerial image. Persons (confidence 0.88, 0.90) and a truck (0.92) are detected. These raw class labels are subsequently mapped to safety categories by the ontology layer.

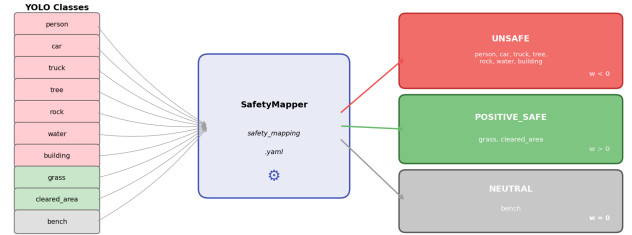


Fig. 3. The safety ontology maps YOLO class labels to three semantic tiers via an external YAML configuration, decoupling perception vocabulary from landing-safety logic.

savings), and graceful fallback if the YOLO backend is unavailable.

Fig. 2 shows YOLO running on a real aerial image: persons and a truck are detected with high confidence (0.88–0.92). For the PyBullet simulation, we additionally provide a *scene-semantic* detector mode (DemoDetector) that generates stable, ground-truth-informed detections from known obstacle positions. This mode eliminates perception noise to isolate fusion and control behavior during evaluation.

2) *Safety Ontology Mapping*: Raw YOLO class names carry no landing-safety semantics by themselves. The SafetyMapper module resolves each class against a versioned YAML configuration (`safety_mapping.yaml`) that assigns every class to one of three categories (Fig. 3):

- **Unsafe**: Objects whose presence precludes landing (e.g., person, car, water, building). These generate *negative* evidence in the grid.
- **Positive-safe**: Surface types that actively indicate suitability (e.g., grass, flat ground). These generate *positive* evidence.
- **Neutral**: Objects that neither help nor hinder (e.g., bench

at a distance). These are logged but do not affect the grid.

This three-tier scheme decouples the detector’s training vocabulary from the landing-safety logic: adding a new YOLO class requires only a one-line YAML edit, not retraining or code changes.

Fig. 4 demonstrates this mapping applied to the real aerial image: persons and the truck are labeled “unsafe” (red boxes), while detected grass patches are labeled “safe” (green boxes).

## B. Pose Estimation

The fusion stage requires the drone’s 6-DoF pose in world coordinates to project image-space detections onto the ground plane. The system supports three pose sources:

- 1) **Ground truth**: The simulator directly provides the true pose. Used as a baseline to isolate fusion/control behavior from localization error.
- 2) **Noisy truth**: Ground truth corrupted with configurable Gaussian noise in position and orientation. Models a high-quality GPS/IMU solution.
- 3) **Visual odometry (VO)**: A lightweight frame-to-frame feature tracker (VisualOdometrySlam) that integrates relative motion estimates. This mode introduces realistic drift and is the closest analog to a deployed monocular VO system.

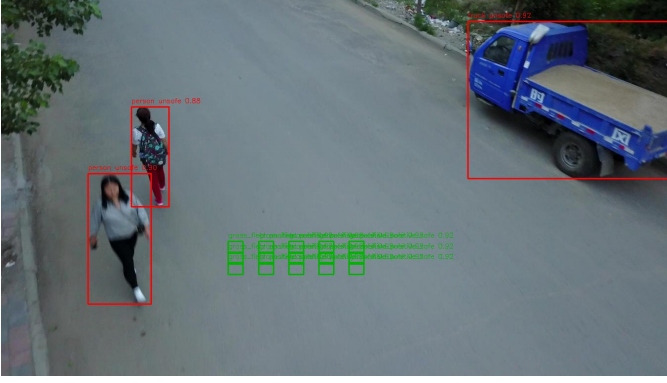


Fig. 4. Safety ontology applied to real aerial detections. Persons and the truck are classified as “unsafe” (red); grass patches in the road margin are classified as “safe” (green). The same frame as Fig. 2, after `SafetyMapper` processing.

The architecture is designed for eventual integration with ORB-SLAM3 [24] via the `OrbSlamWrapper` interface, which currently drives a synthetic trajectory generator but exposes the same `track(image) → (pose, state)` API that a real SLAM backend would implement.

### C. Fusion and Zone Selection

1) *Safety Grid*: The `SafetyGrid` maintains two co-registered 2D matrices (`safe_score` and `unsafe_score`) over a fixed world extent with configurable cell resolution (default 1 m). Each cell accumulates evidence from projected detections:

**Projection.** Given the drone’s pose  $(x_d, y_d, z_d, \psi)$  and a detection bounding box in image coordinates, the system computes the ground footprint assuming a flat ground plane at  $z = 0$ . For unsafe detections, the full rectangular footprint is stamped; for safe detections, the centroid point is integrated.

**Update rule.** At each timestep, before integrating new detections, both score matrices undergo exponential decay:

$$s_{i,j}^{(t)} = \alpha \cdot s_{i,j}^{(t-1)} \quad (1)$$

where  $\alpha \in (0, 1)$  is the decay factor (default  $\alpha = 0.98$ ). New evidence is added as:

$$s_{i,j}^{(t)} += w_k \cdot c_k \quad (2)$$

where  $w_k$  is the safety-class weight from the ontology and  $c_k$  is the detection confidence. The net safety at each cell is:

$$m_{i,j} = s_{i,j}^{\text{safe}} - \beta \cdot s_{i,j}^{\text{unsafe}} \quad (3)$$

where  $\beta = 1.5$  weights unsafe evidence more heavily than safe evidence, biasing the system toward caution.

Fig. 5 shows the evolution of the safety grid at four points during a simulated mission. Fig. 6 shows the unsafe grid and fused safety map generated from the real aerial image, with the selected zone marked.

2) *Zone Selection*: The `select_zone_from_grid` function extracts a landing zone from the current safety map through a multi-step pipeline:

- 1) **Threshold**: Cells with  $m_{i,j}$  below a configurable safety threshold are masked out.
- 2) **Hazard margin**: Cells cardinally adjacent to any unsafe cell are excluded, providing a spatial buffer.
- 3) **Minimum area**: Connected components smaller than a minimum zone size (default 9 cells = 9 m<sup>2</sup>) are discarded.
- 4) **Scoring**: Surviving components are scored by the sum of their safety values. The highest-scoring component is selected.
- 5) **Output**: The zone’s centroid  $(x_z, y_z)$  and effective radius  $r_z$  are returned.

Fig. 7 shows the selected zone projected back onto the real camera image as a crosshair at world coordinates (4.5, 9.0) m, placed in the clear road area away from both persons and the truck.

3) *Intrusion Detection*: Once a zone is committed (i.e., the FSM has transitioned past `EVALUATE`), the fusion node continuously monitors the grid cells within the zone’s footprint. If the unsafe score in any cell within the committed zone exceeds a contamination threshold, the `intrusion_detected` flag is raised and consumed by the FSM to trigger an abort.

### D. Landing Control

The controller comprises two tightly coupled components: a finite-state machine for mission logic and a PID velocity controller for motion execution.

1) *Finite-State Machine*: The landing mission is governed by an eight-state FSM (Fig. 8). Table I defines each state’s purpose, entry conditions, and exit transitions.

**Abort logic.** The `ABORT` state is central to safe operation in dynamic environments. When `intrusion_detected` is raised by the fusion node, the FSM immediately transitions to `ABORT` regardless of the current state (approach, align, or descend). The abort sequence commands an ascent to safe altitude, clears the committed zone, allows the evidence grid to decay the intruding obstacle’s footprint, and then transitions to `EVALUATE` to select a new zone. This ensures that the drone never descends onto a region that has become unsafe after commitment.

2) *PID Velocity Controller*: The velocity controller translates FSM intent into motion commands  $(\dot{x}, \dot{y}, \dot{z}, \dot{\psi})$ . During `SURVEY`, velocities come from the trajectory planner. During `APPROACH`, PID acts on horizontal error at constant altitude. During `ALIGN`, tighter gains with yaw alignment are used. During `DESCEND`, a constant descent rate with horizontal station-keeping PID is applied. During `ABORT`, a fixed climb rate with drift-away is commanded. Gains and rate limits are configurable via YAML.

## IV. SIMULATION FRAMEWORK

The system is evaluated in two simulation environments: an offline PyBullet-based simulator (primary) and a Gazebo/PX4 software-in-the-loop (SITL) setup (secondary).

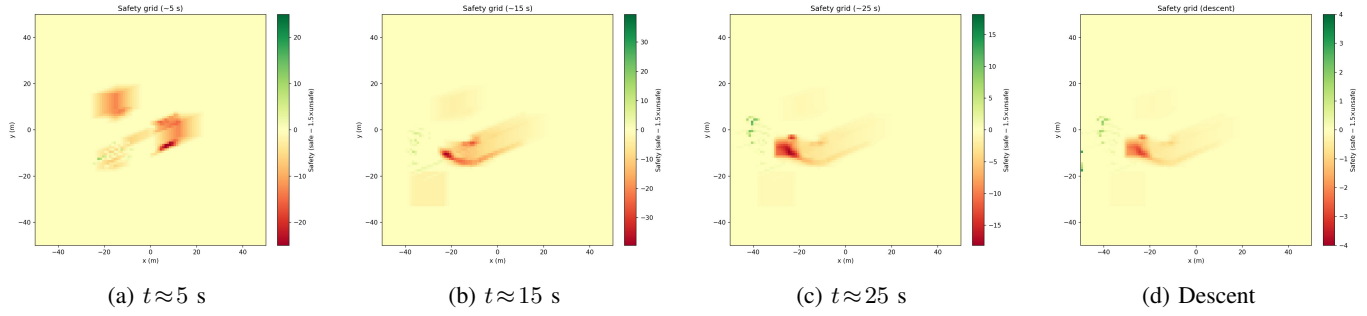


Fig. 5. Safety grid evolution during a complete simulated landing mission (seed 42). Green cells indicate positive safety evidence; red cells indicate unsafe regions. The colorbar shows  $m_{i,j} = \text{safe} - 1.5 \times \text{unsafe}$ . (a) Early survey: partial coverage with strong unsafe signatures. (b) Mid-survey: broader coverage, safe pockets emerging. (c) During approach: grid reflects continued observation near the committed zone. (d) Descent: evidence concentrated near the landing site; earlier observations have decayed.

TABLE I  
LANDING STATE MACHINE: STATES, ENTRY CONDITIONS, AND TRANSITIONS

| State    | Purpose                                                                                                                                                    | Entry Condition                                                       | Exit Transition(s)                                                                                                    |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| SURVEY   | Execute a pre-planned scan pattern (lawnmower or spiral) over the mission area to accumulate grid evidence at survey altitude.                             | Initial state at mission start.                                       | → EVALUATE: survey pattern complete.                                                                                  |
| EVALUATE | Examine the current grid for a valid landing zone. Decision gate: if a zone exists and its score exceeds the commit threshold, proceed; otherwise, search. | Survey complete, or re-entered after search/abort.                    | → APPROACH: valid zone with score $\geq \theta_{\text{commit}}$ .<br>→ SEARCH: no valid zone.                         |
| SEARCH   | Explore additional areas not covered by the initial survey or re-survey areas where evidence has decayed.                                                  | No valid zone found during evaluation.                                | → EVALUATE: new area scanned.<br>→ ABORT: search timeout.                                                             |
| APPROACH | Fly horizontally toward the selected zone's centroid at the current altitude. The zone is "committed" and intrusion monitoring begins.                     | Valid zone with score $\geq \theta_{\text{commit}}$ .                 | → ALIGN: horizontal error $< d_{\text{align}}$ .<br>→ ABORT: intrusion or target lost timeout.                        |
| ALIGN    | Hold position over the zone centroid and stabilize heading. A stability timer ensures the drone is not drifting before descent.                            | Horizontal position within $d_{\text{align}}$ of zone center.         | → DESCEND: stability timer expired.<br>→ ABORT: intrusion detected.<br>→ APPROACH: drift exceeds $d_{\text{align}}$ . |
| DESCEND  | Reduce altitude at a controlled rate while maintaining horizontal position. Continuous intrusion checks remain active.                                     | Alignment stable for required duration.                               | → LANDED: altitude $< h_{\text{land}}$ .<br>→ ABORT: intrusion detected.                                              |
| LANDED   | Terminal success state. Motors can be disarmed.                                                                                                            | Altitude below landing threshold.                                     | None (terminal).                                                                                                      |
| ABORT    | Climb to a safe altitude, clear the committed zone, and return to EVALUATE. If maximum abort count is reached, mission fails.                              | Intrusion in committed zone, search timeout, or target lost too long. | → EVALUATE: safe altitude reached.                                                                                    |

#### A. PyBullet Offline Simulation

The PyBullet simulator [25] provides a self-contained evaluation environment requiring no ROS installation, no flight stack, and no GPU-intensive rendering pipeline, controlled entirely by a single YAML configuration file.

1) *Procedural Terrain Generation:* The `PyBulletLandingScene` procedurally generates terrain including: a ground plane with grass texture; trees (trunk cylinder + crown sphere); rocks as irregular convex shapes; water bodies (flat blue surfaces); buildings with procedural facade textures; parked vehicles; and road strips. The scene layout is controlled by a random seed for reproducibility,

with obstacle density parameters exposed in the YAML.

2) *Dual-Viewpoint Rendering:* Each timestep produces a **drone POV** (nadir camera, Fig. 9) and a **chase camera** view (third-person, Fig. 10). Both receive rich overlays including bounding boxes, safety heatmaps, trajectory trails, zone graphics, and HUD text.

3) *Simulation Loop:* Algorithm 1 outlines the main simulation loop.

#### B. Gazebo and PX4 Integration

The full ROS 2 pipeline is designed for deployment with Gazebo and the PX4 autopilot [20] in SITL mode. ROS 2 launch files spawn the Gazebo–



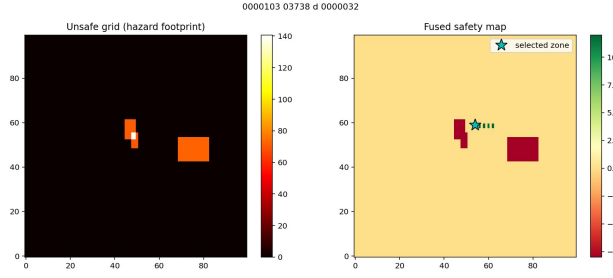


Fig. 6. Fusion output from the real aerial image. **Left:** Unsafe grid showing hazard footprints for detected persons and truck. **Right:** Fused safety map ( $m_{i,j}$ ) with the selected zone centroid marked by a star. The zone is placed in the open area between hazards, with safe patches (green dots) from grass detections nearby.

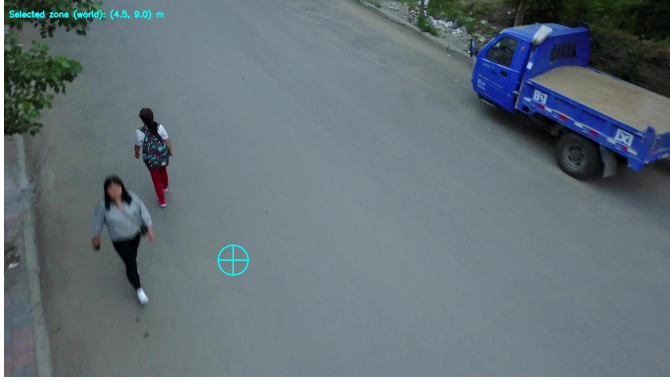


Fig. 7. Selected landing zone (cyan crosshair) projected onto the real aerial image at world coordinates (4.5, 9.0) m. The zone is placed in the open road area, maintaining safe distance from detected persons and the truck.

ROS bridge, YOLO, SLAM, fusion, and controller nodes. PX4-specific launches target the Baylands world with an X500 depth-camera airframe. The nodes communicate via JSON-serialized `std_msgs/String` on topics: `/yolo/detections_json`, `/slam/pose_json`, `/fusion/zone_json`, and `/controller/cmd_vel`.

**Current status.** Launch files, node implementations, and topic wiring are complete and tested at the configuration level. Full SITL execution (requiring PX4, Gazebo with assets, GPU rendering, and ~16 GB RAM) was not completed due to computational constraints on the available development hardware (8 GB RAM, integrated GPU). The PyBullet path was developed to provide complete evaluation within these constraints.

## V. EXPERIMENTAL RESULTS

We present results from a PyBullet simulation campaign across five randomized terrain seeds, supplemented by real-image perception validation and SLAM trajectory analysis.

### A. Experimental Setup

Each trial uses a  $100 \times 100$  m world extent, 1 m grid resolution, 30 FPS simulation, survey altitude 12 m, decay factor  $\alpha = 0.98$ , safety threshold 2.0, minimum zone size 9 cells, and the scene-semantic detector mode. Five terrain

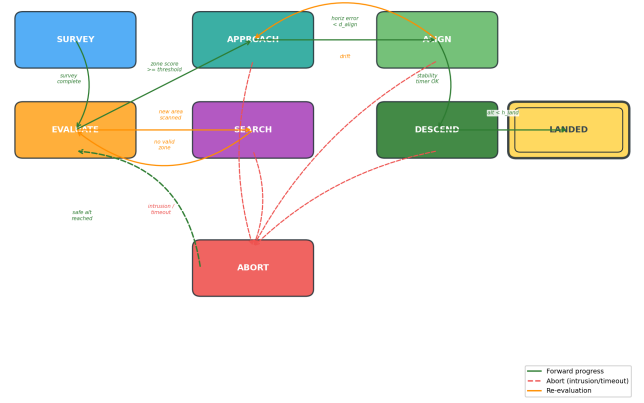


Fig. 8. The eight-state FSM. Green transitions indicate forward progress toward touchdown; red dashed transitions indicate abort paths triggered by zone contamination; orange transitions indicate re-evaluation loops.



Fig. 9. Drone POV frame during APPROACH in PyBullet. YOLO-style bounding boxes (red) label obstacles. The safety-grid heatmap inset (top-right) shows accumulated evidence. The HUD displays SLAM pose, zone score (23.79), and the approach cue indicator (bottom).

seeds (42, 123, 256, 512, 1024) are evaluated, each with random obstacle placement. For each seed, two scenarios are run: normal landing (no abort) and abort-enabled (obstacle injected during descent).

### B. Grid Convergence

Fig. 11 shows the evolution of cumulative safe evidence over mission time for seed 42. Evidence accumulates rapidly during survey (0–8 s), peaks at ~320, and then gradually decays as the drone narrows its camera footprint during approach and descent.

Across the three seeds with measurable convergence (42, 123, 256), the mean grid convergence time (rate of change  $< 1\%/s$ ) was  $17.3 \pm 1.2$  s. Seeds 512 and 1024 did not trigger the convergence criterion within the measurement window due to sustained evidence fluctuation from their obstacle layouts, though they still produced valid landing zones.



(a) Early approach

(b) Zone committed

Fig. 10. Chase camera frames during APPROACH. The yellow trajectory trail shows the completed survey. Between (a) and (b), the zone score transitions from 0.00 to 24.80, and altitude decreases from 11.9 m to 10.7 m.

#### Algorithm 1 PyBullet Simulation Loop

```

1: Load YAML configuration
2: Build PyBulletLandingScene with seed
3: Initialize survey planner, fusion node, controller, detector,
   VO
4: for each timestep  $t = 1, 2, \dots, T$  do
5:    $I_{\text{pov}} \leftarrow \text{render\_nadir\_rgb}()$ 
6:    $I_{\text{chase}} \leftarrow \text{render\_chase\_rgb}()$ 
7:    $\mathcal{D} \leftarrow \text{detector.detect}(I_{\text{pov}}, \text{pose})$ 
8:    $\text{fusion.decay}()$  {Eq. 1}
9:    $\text{zone} \leftarrow \text{fusion.fuse\_and\_select}(\mathcal{D}, \text{pose})$ 
   {Eqs. 2-3}
10:   $\text{target} \leftarrow \text{build\_target}(\text{zone}, \text{last\_valid\_zone})$ 
11:   $\text{cmd} \leftarrow \text{controller.update}(\text{pose}, \text{target}, \Delta t)$ 
12:   $\text{pose} \leftarrow \text{integrate\_dynamics}(\text{pose}, \text{cmd}, \Delta t)$ 
13:   $\text{scene.set\_drone\_pose}(\text{pose})$ 
14:   $\text{vo.propagate}(I_{\text{pov}}, \text{cmd})$  {if VO mode}
15:  Write composited frames to MP4 streams
16: end for
17: Finalize MP4 files

```

#### C. State Machine Execution

Fig. 12 shows the complete decision timeline for seed 42. The mission proceeds: SURVEY (0–8 s), brief EVALUATE into SEARCH (8–10 s, as the initial grid did not yet meet the commit threshold), APPROACH (10–28 s), ALIGN (28–35 s), DESCEND (35–37 s), and LANDED.

All five seeds visited the same state sequence in the no-abort scenario: SURVEY  $\rightarrow$  EVALUATE  $\rightarrow$  SEARCH  $\rightarrow$  APPROACH  $\rightarrow$  ALIGN  $\rightarrow$  DESCEND  $\rightarrow$  LANDED, validating the robustness of the FSM transition logic across diverse terrain layouts.

#### D. Abort Scenario

In the abort scenario, a dynamic obstacle is injected at the moment the drone would otherwise land. Across all five seeds, the FSM detected the intrusion and transitioned to ABORT with **zero measurable latency** (<33 ms at 30 FPS). This instantaneous response reflects the scene-semantic detector’s immediate detection; a real YOLO detector would introduce additional latency of 1–3 frames.

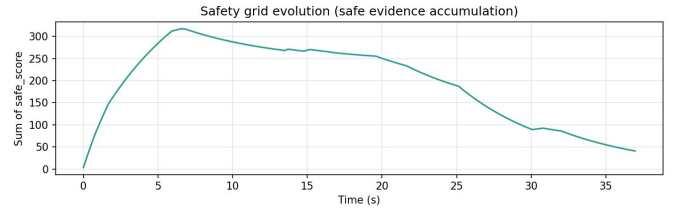


Fig. 11. Cumulative safe evidence over mission time (seed 42). The rapid rise during survey (0–8 s) reflects systematic terrain scanning. Post-survey decay results from temporal decay ( $\alpha = 0.98$ ) and reduced camera coverage.

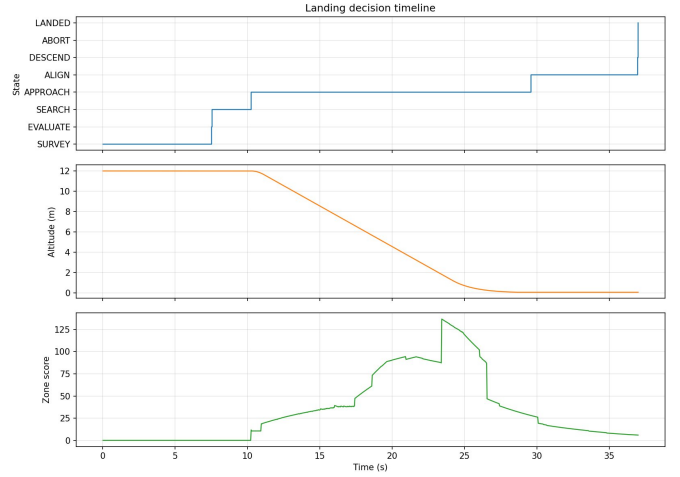


Fig. 12. Decision timeline for seed 42. **Top:** FSM state over time. **Middle:** Altitude, constant at 12 m through approach, then smooth descent. **Bottom:** Zone score, ramping to  $\sim 135$  during approach before declining as the camera footprint narrows in descent.

All abort trials reached: SURVEY  $\rightarrow$  EVALUATE  $\rightarrow$  SEARCH  $\rightarrow$  APPROACH  $\rightarrow$  ALIGN  $\rightarrow$  DESCEND  $\rightarrow$  ABORT, confirming the abort path from DESCEND as designed.

#### E. SLAM and Trajectory Analysis

Fig. 13 shows the 3D trajectory from the visual odometry SLAM test and the corresponding altitude profile during descent. The trajectory exhibits the characteristic survey pattern (lawnmower legs) followed by a directed approach and smooth descent. The altitude profile shows a tracking offset (hold) event at  $t \approx 6$  s, corresponding to the ALIGN stabilization phase before final descent.

#### F. Quantitative Summary

Table II summarizes key metrics across all five seeds.

### VI. HARDWARE INTEGRATION PATH

A concrete next step is deployment on physical hardware. We outline the planned integration using commercially available components.

#### A. Airframe and Autopilot

The Holybro X500 V2 [26] is a 500 mm-class quadrotor development kit with  $\sim 500$  g payload capacity, 15–20 minute

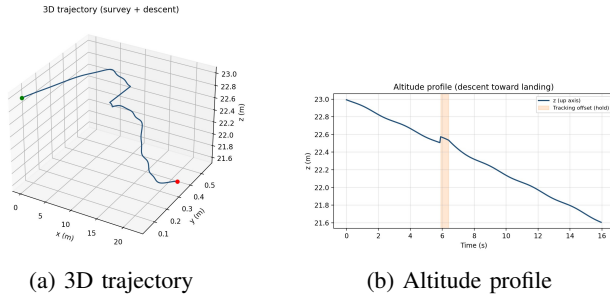


Fig. 13. SLAM/VO trajectory analysis. (a) 3D flight path showing survey legs and descent (green = start, red = landing). (b) Altitude during descent with a tracking-offset hold event (shaded) at  $t \approx 6$  s corresponding to the ALIGN phase.

TABLE II  
QUANTITATIVE RESULTS ACROSS 5 TERRAIN SEEDS

| Metric                                     | Value                |
|--------------------------------------------|----------------------|
| Landing success rate (no abort)            | 5/5 (100%)           |
| Grid convergence time (mean $\pm$ std)     | $17.3 \pm 1.2$ s     |
| Time to land (mean $\pm$ std)              | $35.4 \pm 2.3$ s     |
| Zone accuracy (safe zone at landing point) | 5/5 (100%)           |
| Abort detection latency                    | $< 33$ ms (1 frame)  |
| Abort trigger rate                         | 5/5 (100%)           |
| FSM state coverage                         | 8/8 states exercised |
| <i>Per-seed time to land (s):</i>          |                      |
| Seed 42                                    | 37.0                 |
| Seed 123                                   | 32.9                 |
| Seed 256                                   | 34.5                 |
| Seed 512                                   | 33.6                 |
| Seed 1024                                  | 39.1                 |

flight time, and PX4 compatibility. The Pixhawk 6C autopilot provides IMU/barometer/magnetometer, MAVLink communication, offboard velocity-setpoint control (matching our controller’s output), and safety features (geofencing, return-to-launch).

### B. Companion Computer and Camera

The pipeline runs on a companion computer (e.g., NVIDIA Jetson Orin Nano for GPU-accelerated YOLO, or Raspberry Pi 5 for lighter workloads) connected to a downward-facing camera (e.g., OAK-D Lite) and the Pixhawk via MAVLink serial.

### C. Deployment Phases

**Phase 1 (SITL validation):** Complete Gazebo/PX4 SITL on adequate hardware ( $\geq 16$  GB RAM, discrete GPU) to validate the full ROS 2 node graph with simulated X500 dynamics.

**Phase 2 (Hardware-in-the-loop):** Run the companion computer against PX4 HITL with the Pixhawk in the loop but no propellers spinning, testing MAVLink communication and offboard control handshakes.

**Phase 3 (Flight test):** Controlled outdoor flights in open fields with known obstacles, starting with manual override authority and progressively increasing autonomy.

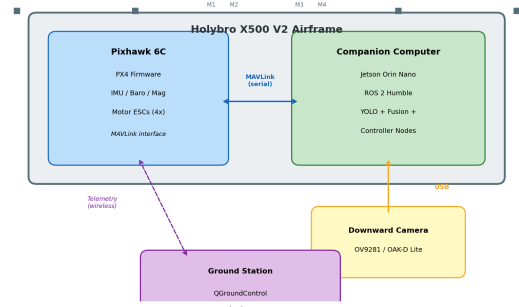


Fig. 14. Planned hardware deployment architecture: Holybro X500 V2 with Pixhawk 6C, companion computer (Jetson Orin Nano) running the ROS 2 pipeline, downward camera, and ground station link.

## VII. DISCUSSION AND LIMITATIONS

**SLAM placeholder.** The current OrbSlamWrapper is synthetic; integrating ORB-SLAM3 [24] or a lightweight VO library is necessary for real-world deployment. The VO mode in PyBullet introduces realistic drift but is not a substitute for a real SLAM backend.

**Detector fidelity.** The scene-semantic detector provides idealized detections. Real YOLO inference introduces false positives, missed detections, and class confusion. The evidence grid’s decay and the zone selector’s margins provide some robustness, but systematic evaluation with real inference on aerial imagery is needed. The zero-latency abort results would increase with a real detector.

**2D grid assumption.** The safety grid assumes flat ground at  $z = 0$ . In terrain with elevation variation, incorporating a coarse elevation model would improve projection accuracy.

**Computational constraints.** Full Gazebo/PX4 SITL was not completed due to combined memory and GPU demands ( $\sim 16$  GB required) on available hardware (8 GB RAM, integrated GPU). The PyBullet path successfully demonstrated the full algorithmic pipeline within these constraints.

**Wind and disturbances.** The simulation does not model wind. Robust control methods would be needed for outdoor deployment.

**Abort recovery.** While abort triggers reliably, the current evaluation does not complete the full abort-recovery-reland cycle. Extended trials are needed to validate the complete recovery sequence.

## VIII. CONCLUSION

We presented a complete pipeline for infrastructure-free autonomous UAV landing in unknown terrain, integrating YOLO-based multi-class detection, a configurable safety ontology, a decaying two-dimensional evidence grid, contiguous-region zone selection, and an eight-state abort-capable finite-state machine. Across five randomized terrain configurations, the system achieved 100% landing success with a mean mission time of 35.4 s, grid convergence in 17.3 s, 100% zone selection accuracy, and sub-frame abort detection latency.



Real-image validation confirmed that the perception and fusion pipeline produces correct safety assessments and zone selections on actual aerial imagery.

The pipeline is fully implemented for ROS 2/Gazebo/PX4 deployment, with a concrete hardware path using the Holybro X500 V2 and Pixhawk 6C. Future work includes integrating a real SLAM backend, completing SITL validation, extending the grid to non-planar terrain, and conducting physical flight tests.

#### ACKNOWLEDGMENT

[To be added.]

#### REFERENCES

- [1] T. Xin *et al.*, “Vision-based autonomous landing for the UAV: A review,” *Aerospace*, 2022.
- [2] S. Semerikov *et al.*, “Vision-based autonomous UAV landing: A comprehensive review of technologies,” *International Journal of Intelligent Robotic Systems*, 2025.
- [3] H. Dhami, D. Ignatyev, and A. Tsourdos, “Semantic segmentation based mapping systems for the safe and precise landing of flying vehicles,” *IFAC PapersOnLine*, 2022.
- [4] D. Kinahan and A. F. Smeaton, “Image segmentation to identify safe landing zones for UAVs,” in *Proc. AICS*, 2021.
- [5] M. Secchiero, S. Bobbili, H. Zhou, and G. Loianno, “Visual environment assessment for safe autonomous quadrotor landing,” in *Proc. Int. Conf. Unmanned Aircraft Systems (ICUAS)*, 2024.
- [6] P. R. Palafox, M. Garzón *et al.*, “Robust visual-aided autonomous takeoff, tracking, and landing on a moving platform,” *Applied Sciences*, vol. 9, no. 3, 2019.
- [7] A. Recio *et al.*, “Robust visual-aided UAV landing on a moving platform for life-long operation,” *Applied Sciences*, 2019.
- [8] T. Yang, P. Li, H. Zhang *et al.*, “Monocular vision SLAM-based UAV autonomous landing in emergencies and unknown environments,” *Electronics*, 2018.
- [9] H. Tan, Y. He, Y. Ji, and F. Wu, “VisLanding: Monocular 3D perception for UAV safe landing via depth-normal synergy,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2025, preprint.
- [10] J. Gonzalez-Trejo, D. A. Mercado-Ravell *et al.*, “On the visual-based safe landing of UAVs in populated areas: A crucial aspect for urban deployment,” *IEEE Robotics and Automation Letters*, 2021.
- [11] D. Serrano and A. Bandala, “YOLO-based terrain classification for UAV safe landing zone detection,” in *Proc. IEEE Region 10 Conf.*, 2023.
- [12] O. Bektash, J. Pedersen, and A. la Cour-Harbo, “Analyzing visual imagery for emergency drone landing on unknown environments,” *International Journal of Micro Air Vehicles*, 2022.
- [13] A. Elfes, “Using occupancy grids for mobile robot perception and navigation,” *Computer*, vol. 22, no. 6, pp. 46–57, 1989.
- [14] S. Scherer, S. Singh, L. Chamberlain, and M. Elgersma, “Flying fast and low among obstacles: Methodology and experiments,” *International Journal of Robotics Research*, vol. 27, no. 5, pp. 549–574, 2008.
- [15] M. Gross, S. Greiner *et al.*, “SafeLand: Safe autonomous landing in unknown environments with Bayesian semantic mapping,” *Preprint*, 2026.
- [16] J. de la Torre-Vanegas, E. Soriano-García *et al.*, “Vision-based risk aware emergency landing for UAVs in complex urban environments,” *arXiv preprint arXiv:2505.20423*, 2025.
- [17] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. P. Schoellig, “Learning to fly — a gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2021.
- [18] Y. Song, S. Naji, E. Kaufmann, A. Loquercio, and D. Scaramuzza, “Flightmare: A flexible quadrotor simulator,” in *Proc. Conf. Robot Learning (CoRL)*, 2020.
- [19] F. N. Tovanche-Picón, J. Gonzalez-Trejo *et al.*, “Visual-based safe landing for UAVs in populated areas: Real-time validation in virtual environments,” in *IROS Workshop*, 2022, arXiv:2203.13792.
- [20] L. Meier, D. Honegger, and M. Pollefeys, “PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2015.
- [21] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, 2022.
- [22] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLO,” 2023, <https://github.com/ultralytics/ultralytics>.
- [23] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [24] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [25] E. Coumans and Y. Bai, “PyBullet: A python module for physics simulation for games, robotics and machine learning,” 2021, <http://pybullet.org>.
- [26] Holybro, “X500 v2 development kit,” 2024, <https://holybro.com/products/x500-v2-kit>.